

func_vehicle

How to implement func_vehicle in Copper 1.30

FTEQCC (or equivalent) compiler is required.

func_vehicle comes with 10 .qc files, one of them being **maths.qc** which comes from an earlier version of Copper. Skip that one, and leave the one that comes with Copper instead!

Add the 9 other files into **progs.src** between maths.qc and utility.qc, and a COPPER define, looking like this:

```
...
#pragma autoprot
//#define NO_CSQC 1 // uncomment for a CSQC-less build
#define COPPER      // func_vehicle: skips forward declarations

defs.qc
constants.qc

maths.qc

debug.qc

phys_defs.qc
phys_math.qc
phys_matrix.qc
phys_tetra.qc
phys_exec.qc

phys_object.qc

func_vehicle.qc
func_vehicle_builtins.qc

utility.qc
subs.qc
...
```

Now we have the vehicle stuff included in the game, next we need to add interactions between the vehicle(s) and the rest.

First we want to add some functionality to **client.qc** for driving and entering/exiting the vehicle. In `PutClientInServer`, right below the `setmodel` stuff, add this:

```
...
    setmodel (self, "progs/player.mdl");
    modelindex_player = self.modelindex;

    stuffcmd(self, "chase_active 0\n");
    stuffcmd(self, "crosshair 1\n");

    setsize (self, VEC_HULL_MIN, VEC_HULL_MAX);
...
```

This resets the chasecam if a player stops playing while seated in a vehicle.

Still in **client.qc**, move down to `PlayerPreThink`, and add the following:

```
...
    if (self.deadflag == DEAD_DYING)
        return;        // dying, so do nothing

    if (self.driving)
        return;

    self.wait = 0;
...
```

Next, move down to `PlayerPostThink`. First we need to add this:

```
...
    // do weapon stuff
    if (self.driving)
        ImpulseCommands();
    else
        W_WeaponFrame ();
...
```

And add this under `CheckPowerups`:

```
...
    CheckPowerups ();
```

When the player dies, they need to get tossed out from the vehicle. Open up **player.qc** and navigate to PlayerDie to add this:

```
...
void() PlayerDie =
{
    local float i;

    if (self.driving)
        Car_AttachToVehicle(self, self.driving);

    self.items = self.items - (self.items & IT_INVISIBILITY);
    ...
}
```

Next up we need to add a new impulse so players can enter and exit nearby vehicles. Open up **impulse.qc** and navigate to ImpulseCommands to add the following:

```
...
case 15:
    W_SelectBestWeapon();
    break;
case 50:
    Car_CheckAttach();
    break;
case 40:
    dev_impulse_list();
    break;
...
}
```

You've successfully added the basic functionality for entering, exiting and driving vehicles!

In **ai.qc**, to let monsters detect players in cars, put this into visible:

```
...
if (trace_inopen && trace_inwater)
    return FALSE; // sight line crossed contents

if (trace_fraction < 1 && trace_ent.owner.driving == targ)
    return TRUE;

if (trace_fraction == 1)
    ...
}
```

Still in <

The last change in **ai.qc** is in FilterTarget. Copper makes monsters ignore players using noclip, but players driving a vehicle use MOVETYPE_NOCLIP too:

```
...
float(entity client) FilterTarget =
{
    if (client.flags & FL_NOTARGET || (client.movetype == MOVETYPE_NOCLIP
    && !client.driving))
        return FALSE;
    ...
}
```

The Enforcer and the Scrag do their own check for a clear shot. In **m_enforcer.qc**, change this line in **enforcer_clear_attack**:

```
...
    if (trace_inopen && trace_inwater) return FALSE;    // sight line
crossed contents
    if (trace_ent != self.enemy && trace_ent.owner.driving != self.enemy)
return FALSE; // don't have a clear shot
    return TRUE;
...
}
```

In **m_wizard.qc**, change this in WizardCheckAttack:

```
...
    traceline2(spot1, spot2, self, TRACE_WATER);
    if (trace_ent != self.enemy && trace_ent.owner.driving != self.enemy)
    {
        // don't have a clear shot, so move to a side
    }
    ...
}
```

If you want shambler lightning to do damage, open up **m_shambler.qc** and find the CastLightning function and change the following:

```
...
    ClearMultiDamage();
    LightningBeam (org, trace_endpos + dir*4, self, 10);
    ApplyMultiDamage();
...
```

And you're done!

One extra detail: if you want vehicles to spawn faster, and don't mind breaking compatibility with older engines, you can remove the quote slashes on the #define QUICKSPAWN line in **phys_defs.qc**

/ Marko "Polo" Permanto